

Local AI Orchestrator

Onboarding — how to use it

Practical, command-by-command guide. Pair with the System Guide for the why and how.

Generated: 2026-05-17

Quick reference

Every common operation, one row. Run all commands from the project root: `cd /Users/macbookpro/Projects/local-ai-orchestrator`.

Goal	Command
See system health (Docker, Ollama, ports)	<code>bash scripts/check_system.sh</code>
Start Docker services	<code>bash scripts/start_services.sh</code>
Start Ollama (persistent service)	<code>brew services start ollama</code>
Start the control panel	<code>bash scripts/start_control_panel.sh</code>
Open the panel in browser	<code>http://127.0.0.1:8501</code>
Create a work packet from CLI	<code>bash scripts/create_work_packet.sh "Title" "Description"</code>
Submit answers to clarification	<code>bash scripts/answer_questions.sh <id> examples/sample_answers.md</code>
Allow local models in the day	<code>bash scripts/day_unlock.sh "reason"</code>
Lock the day-unlock again	<code>bash scripts/day_lock.sh</code>
Load/unload/test a model in browser	Panel → Models tab
Edit day/night times in browser	Panel → Schedule tab
Verify nothing runs automatically	<code>launchctl list grep localai (empty = off)</code>
Pull all configured models (~41 GB)	<code>bash scripts/pull_models.sh</code>
Stop Docker services (keep data)	<code>bash scripts/stop_services.sh</code>
Run the full test suite	<code>bash scripts/run_tests.sh</code>

Daily start sequence

Every time you want to use the system, run these four steps in order. None of them load models or do heavy work; they just bring services online.

```
cd /Users/macbookpro/Projects/local-ai-orchestrator

# 1. Make sure Ollama is running (persistent service)
brew services start ollama
curl http://localhost:11434/api/tags      # should return JSON

# 2. Start Docker services (Postgres, Temporal, Open WebUI)
bash scripts/start_services.sh

# 3. Confirm everything is up
bash scripts/check_system.sh

# 4. Start the control panel
bash scripts/start_control_panel.sh
# then browse to http://127.0.0.1:8501
```

All four are safe to re-run. `start_services.sh` skips already-running containers; `brew services start ollama` is a no-op if it's already loaded; `start_control_panel.sh` will fail if port 8501 is held by a previous Streamlit (kill it first with `kill $(lsof -nP -iTCP:8501 -sTCP:LISTEN -t)`).

A complete worked example

End-to-end use of the system today, from a vague request to a ready packet. Execution is still scaffolded; this stops before the night step.

1. Create a vague packet (expect rejection)

```
bash scripts/create_work_packet.sh "Tomorrow prep" \  
  "Prepare me for tomorrow from my notes."
```

Expected output:

```
{  
  "work_packet_id": "abc123...",  
  "status": "UNDERDEFINED",  
  "readiness_score": 0.18,  
  "questions": [  
    {"category": "objective",      "question": "What exact outcome..."},  
    {"category": "outputs",        "question": "What output files..."},  
    {"category": "sources",        "question": "Which folders are..."},  
    {"category": "privacy/cloud",  "question": "Is cloud allowed..."},  
    ...  
  ]  
}
```

The score is below 0.65 and status is UNDERDEFINED. **This is correct.** The system refuses to act until you answer the gating questions.

2. Look at the sample answer format

```
cat examples/sample_answers.md
```

3. Edit your own answers or reuse the sample

Write a markdown file with answers. Specific paths under ~/LocalAI/inbox/, explicit cloud policy, clear quality bar, named audience.

4. Submit the answers

```
bash scripts/answer_questions.sh abc123... examples/sample_answers.md
```

Expected output:

```
{  
  "work_packet_id": "abc123...",  
  "status": "READY_FOR_OVERNIGHT",  
  "readiness_score": 0.90,  
  "database": "updated",  
  "remaining_questions": []  
}
```

Status flipped, no remaining questions. The packet is eligible for overnight execution.

Today, the actual overnight execution is scaffolded. Submitting overnight runs creates an `execution_runs` row and queues a Temporal workflow that returns a placeholder. Wiring the real pipeline is the next major chunk of feature work.

Using the Models tab

From <http://127.0.0.1:8501>, click **Models**. You'll see three sections:

Currently loaded (in VRAM)

What Ollama has resident right now. Each row shows the tag, VRAM size, and an expires_at timestamp (when Ollama's keep-alive timer will auto-unload). An **Unload** button frees VRAM immediately.

Pulled models (on disk)

What's available to load. Each row shows the tag, the model group it maps to, size, and modification time. **Load** pulls weights into VRAM with a 30-minute keep-alive. Buttons are disabled in strict DAY_MODE; if you need to load during the day, run `bash scripts/day_unlock.sh "reason"` first.

Quick test prompt

Single-shot verification. Pick a loaded model, type a prompt, hit Send. The response renders in a code block. Not a chat — no history. For conversations, use Open WebUI at <http://localhost:3000>.

First inference after a fresh Load is always the slowest (KV cache cold). Subsequent prompts are 5-10x faster. With two 30B models loaded at once (61.6 GB VRAM), expect noticeable slowdown from memory pressure on a 96 GB M2 Max.

Using the Schedule tab

From the panel, click **Schedule**. Two sections:

Status metrics

Three independent checks of the launchd job: is the project plist present, is the plist installed in ~/Library/LaunchAgents, is it loaded in launchctl. All three off = auto-execution is OFF.

Edit times

Three editable fields: day_mode_start, night_mode_start, night_mode_end. Format 24-hour HH:MM. Click Save to rewrite config/assistant.yaml atomically. Validation runs first; bad values are rejected with a clear error.

Editing the times does NOT auto-flip LOCALAI_MODE. That stays an explicit env-var gesture. The times drive (a) the launchd plist when you regenerate it, and (b) the visual hint in the header showing whether you're in DAY or NIGHT window right now.

If the plist is already installed

The panel shows a yellow warning with the three commands needed to regenerate and reload the plist so the new times take effect. The panel never touches launchd directly — arming and disarming are deliberate terminal commands.

Day-unlock switch

DAY_MODE blocks every local-* model group. To deliberately allow local models during the day, the day-unlock switch flips a single, visible boolean.

Turn it on

```
bash scripts/day_unlock.sh "checking last night's output"
```

Creates ~/LocalAI/state/day_unlock.flag with a timestamp, hostname, and reason. The Streamlit header flips Day unlock from off to ACTIVE and a red banner appears with the flag contents.

Turn it off

```
bash scripts/day_lock.sh
```

Deletes the flag. Warns if LOCALAI_DAY_UNLOCK is also set in the shell so you know to unset it.

One-shot env-var variant

```
LOCALAI_DAY_UNLOCK=true .venv/bin/python -m assistant_core.cli sample-readiness
```

Per-shell, ephemeral. Useful for one-off commands without touching the flag file.

Day unlock does NOT open the cloud path. cloud-claude-opus still raises a `SafetyError` regardless. Full design rationale: [docs/day_unlock.md](#).

Do / Don't

Do

- Keep input files under `~/LocalAI/inbox/`. That's the only place this system reads from.
- Be specific in clarification answers. Explicit paths, single-line `key:value` lines parse better than prose.
- Mark packets as `high_stakes` (board, CEO, legal, investor) when they need the 0.90 readiness bar.
- Use `day_unlock` deliberately, lock again when done. The red banner is your reminder.
- Run `bash scripts/run_tests.sh` before committing changes.
- Read `AGENTS.md` if you're going to modify this repo.
- Pull models on stable Wi-Fi; `qwen3:30b-a3b` is ~18 GB.

Don't

- Don't put `ANTHROPIC_API_KEY` in `.env` unless you actually want cloud fallback. Default policy refuses it; setting the key removes one of six gates.
- Don't point the system at `~/Documents`, `~/Desktop`, or anywhere outside `~/LocalAI/inbox/`.
- Don't delete the `temporal_data` Docker volume thinking it holds your work. Your packets live in `postgres_data`.
- Don't edit files in `~/LocalAI/output/` and expect them to come back next run. Save important outputs into your Obsidian vault.
- Don't leave `day_unlock` on overnight if you don't need to. It persists across reboots by design.
- Don't bypass `assert_model_allowed()` by hitting Ollama/LiteLLM directly with raw tags. That defeats the safety policy.
- Don't start the Temporal worker before docker services are healthy.

Troubleshooting

Symptom	Fix
Safari can't connect to 127.0.0.1:8501	Streamlit isn't running. bash scripts/start_control_panel.sh. If port is held: kill \$(lsof -nP -t -i:8501)
Error: could not connect to ollama server	Ollama isn't running. brew services start ollama. Verify with curl http://localhost:11434/api
Docker shows Temporal Restarting	docker logs --tail=20 localai-temporal. If dynamic-config error, force-recreate: docker container rm localai-temporal; docker container create --name localai-temporal --rm
Postgres unavailable	Docker isn't running or postgres container is down. bash scripts/start_services.sh
Day unlock banner won't go away	Refresh the browser. Check echo \$LOCALAI_DAY_UNLOCK in the shell that started Streamlit
bash scripts/install_core.sh fails on Python 3.13	Use Python 3.13: brew install python@3.13 then LOCALAI_PYTHON_BIN=/opt/homebrew/opt/python@3.13/bin
Model pull says "file does not exist"	Tag name wrong. Try alternatives listed at the end of scripts/pull_models.sh, or qwen2.5
Panel Unload doesn't unload	Refresh page. If still loaded, run ollama stop <tag> from terminal as a workaround.
Tests fail with state-dir errors	tests/conftest.py should isolate. If you've deleted it, restore it.

Safety rules at a glance

All enforced in code, not just documented.

Rule	Where enforced
No cloud calls without 6 policy gates passing	assert_cloud_allowed
No local model calls in DAY_MODE by default	assert_heavy_execution_allowed + LOCAL_MODEL_GROUPS
Day unlock is the only relax mechanism, and it's visible	day_unlock_active + Streamlit banner
No writes outside allowed roots	assert_original_file_write_allowed
No emails / external API writes in v1	assert_no_external_write (always denies)
Cloud Claude blocked at model selection too	assert_model_allowed('cloud-claude-opus', ...) raises
Services bind to 127.0.0.1 only	docker-compose.yml port mappings
Original inbox files never modified	Output paths constrained to ~/LocalAI/output/
Nightly auto-execution OFF by default	launchd plist generated but not installed; arming is a 3-step manual g

One paragraph to remember

A vague request comes in. The system refuses to act on it. Instead it asks you five to seven questions sized to the gaps in the request. You answer them in a markdown file. The system rescores. If readiness is ≥ 0.85 (or ≥ 0.90 for high-stakes), the packet is eligible for overnight execution. Overnight, in NIGHT_MODE, the execution graph reads sources, calls local models through a safety gate, evaluates the output, retries on a secondary local model if needed, writes results to `~/LocalAI/output/<date>/`, and adds memory candidates to the Obsidian review queue. Cloud is never used unless six specific conditions are explicitly met. You read the output in the morning. Nothing leaves your machine.

End of Onboarding. For the deeper architecture, see the companion System Guide PDF.